Semester Team Project: Mini Operating System Services Simulator (MOSS)

Course

COSC 414 – Operating Systems (Undergraduate) COSC 514 – Operating Systems (Graduate)

Project Overview

In this semester-long project, you will work in a **team of 2–4 students** to design and implement a **Mini Operating System Services Simulator (MOSS)**. MOSS is a user-space platform that allows users to **experiment with and compare operating system policies** (e.g., scheduling, paging, synchronization) and observe their effects on performance, correctness, and resource usage.

The goal of this project is to help you apply core operating system concepts—such as process management, CPU scheduling, memory management, synchronization, and protection—in a practical, hands-on manner while reasoning about OS design trade-offs.

This project is intentionally divided into **two parts**:

- **Part I:** Individual subsystem development (individual accountability)
- **Part II:** Full system integration (team collaboration)

The simulator will run in **user space on Linux (Ubuntu)** and does **not** require kernel modification. You will implement OS concepts using standard programming languages and libraries while faithfully modeling operating system behavior.

Team Formation

- Team size: **2 to 4 students**
- Each team member must take ownership of **one subsystem** in Part I
- All members are responsible for **integration, testing, and final delivery** in Part II

To encourage cross-learning and prevent siloed work:

- Each team must conduct **at least one cross-review**, where a team member reviews and provides feedback on another member's subsystem.
- Evidence of cross-review (comments, pull requests, or review notes) must be included in the final submission.

Individual responsibility and collaboration must be clearly documented.

Part I – Individual Subsystem Development (Weeks 5–9)

In Part I, each student will independently design and implement **one subsystem** of the OS simulator. Before coding begins, each team must jointly produce a **Subsystem Interface Specification** that defines:

- Public APIs for each subsystem
- Shared data structures
- Assumptions and constraints

This interface specification will be graded and must be approved before implementation proceeds. All subsystems **must conform to this specification** so that integration in Part II is feasible.

Each student must submit **their own code, documentation, and demo** for Part I.

Subsystem Options

Each team member must select **one** of the following subsystems:

Subsystem A: Process Management & CPU Scheduling

Related Topics: Processes, CPU Scheduling

Responsibilities:

- Simulate process creation and termination
- Maintain process control blocks (PCB)
- Implement at least **two CPU scheduling algorithms**:
    - FCFS (First-Come, First-Served) – required
    - Round Robin – required
    - *(Graduate students: add Priority or Multilevel Feedback Queue)*

Required Features:

- Ready queue representation
- Scheduling timeline or Gantt-style output
- Calculation of waiting time and turnaround time
- Handling of edge cases (e.g., empty ready queue, simultaneous arrivals)

Subsystem B: Memory Management & Virtual Memory

Related Topics: Virtual Memory, Paging

Responsibilities:

- Simulate logical to physical address translation using a fixed page size
- Implement page replacement algorithms:
    - FIFO – required
    - LRU – required
    - *(Graduate students: add Optimal or Working Set)*

Required Features:

- Fixed number of physical frames
- Configurable page size and logical address width
- Page fault tracking
- Page replacement visualization (text-based is sufficient)

Subsystem C: Synchronization & Protection

Related Topics: Synchronization, Security, Protection

Responsibilities:

- Implement synchronization mechanisms:
    - Mutex locks
    - Semaphores
- Simulate at least one classical synchronization problem:
    - Producer–Consumer
    - Readers–Writers

Protection Component:

- Basic access control model focused on **conceptual correctness**, not production-grade security
- User roles (e.g., admin, user)
- Permission checks before accessing shared resources

*(For 2 or 4 person teams, this subsystem may be shared or simplified with instructor approval.)**

Part I Deliverables (Individual)

Each student must submit:

- Source code for their subsystem
- Design document (2–3 pages)
- Short demo video (3–5 minutes) demonstrating **1–2 representative scenarios**
- Individual reflection describing design decisions, assumptions, and limitations

---

Part II – Full System Integration (Weeks 10–15)

In Part II, your team will integrate all individual subsystems into **one unified OS simulator**. To reduce integration risk, the integration phase includes **two checkpoints**:

- **Mid-Integration Checkpoint:** Partial integration of at least two subsystems
- **Final Integration:** Fully integrated system

This phase emphasizes **team collaboration, system design, and integration challenges**, similar to real-world operating system development.

Integration Requirements

- Unified command-line interface for the simulator
- Subsystems must interact through well-defined APIs
- System-wide logging and error handling
- Correct enforcement of:
    - CPU scheduling decisions
    - Memory constraints
    - Synchronization rules
    - Access control policies

At least one system-level error scenario must be handled and demonstrated (e.g., deadlock detection, invalid memory access, or permission violation).

Example Commands

```
create_process
schedule RR
access_memory 0x1AF3
lock resource1
```

Vertical Slice Milestone (Required)

To reduce late-stage integration risk, each team must complete a **Vertical Slice Milestone** before full integration.

What is a Vertical Slice?

A vertical slice demonstrates **one process flowing end-to-end through multiple subsystems**, even if the implementation is partial or simplified.

At minimum, the vertical slice must show:

1. A process being created
2. The process being scheduled by the scheduler
3. At least one memory access handled by the memory subsystem
4. At least one synchronization or protection check applied

This milestone ensures that subsystems are **integrated vertically**, not just developed in isolation.

Deliverables for Vertical Slice

- Short demo (2–3 minutes)
- Brief explanation of subsystem interactions
- Known limitations and planned fixes

Failure to complete a working vertical slice on time may result in reduced Part II credit.

Part II Deliverables (Team)

- Integrated system source code
- System architecture diagram
- Final project report (8–12 pages)
- Final demo (live or recorded)
- Peer evaluation form

Grading Breakdown

Component	Weight
Part I – Individual Subsystem	40%
Part II – Integrated System	35%
System Design & Architecture	10%
Documentation Quality	5%

Component	Weight
Presentation & Demo	10%

Expectations by Course Level

COSC 414 (Undergraduate):

- Focus on correct implementation and conceptual understanding
- Qualitative performance discussion
- Clean, well-documented code

COSC 514 (Graduate):

- Either additional algorithms **or** deeper experimental/analytical evaluation
- Quantitative analysis and performance evaluation
- Research-style discussion and design justification

Platform & Tooling Requirements

To ensure consistency, fairness, and smooth integration across teams, all projects **must follow the platform and tooling requirements below**.

Required Platform

- **Operating System:** Ubuntu Linux **22.04 LTS** (required)
  - Must run inside VirtualBox or on a native Linux installation
  - Windows and macOS are not supported as primary development platforms

Programming Languages

- **COSC 414:** C or C++

Compilers & Tools

- GCC / G++ (version 11 or later)
- Make or CMake for builds
- Standard POSIX libraries only (no external OS simulation frameworks)

Version Control

- **Git is required** for all teams
- A shared repository (GitHub or GitLab) must be used
- Meaningful commit messages and incremental commits are expected

Execution Environment

- The simulator must run from the **command line**
- Programs must compile and run using provided build instructions
- Optional: Docker may be used, but native Ubuntu compatibility is still required

Failure to follow platform requirements may result in integration issues and grading penalties.

API Guidelines (Required)

This project is a **modular systems project**, not a collection of standalone programs. All subsystems must communicate through **clean, well-defined APIs**.

General API Principles

- APIs must be **stable**, **minimal**, and **documented**
- Subsystems should expose **function calls**, not internal data structures
- Global variables shared across subsystems are discouraged
- All API calls must include basic error handling

Naming Conventions

- Use clear, descriptive function names
- Prefix functions by subsystem:
  - `sched_` for scheduling
  - `mem_` for memory management
  - `sync_` for synchronization

Input / Output Rules

- Functions must clearly define:
  - Inputs (parameters)
  - Outputs (return values)
  - Possible error conditions
- Do not print directly inside core API functions
- Return status codes and let the main system handle output and logging

Error Handling

- Use consistent return values:
  - `0` for success
  - Negative values for errors (e.g., `-1`, `-2`)
- Errors must be propagated upward, not silently ignored

Ownership & State Management

- Each subsystem owns its internal state
- Other subsystems must not directly modify internal data structures
- State changes must occur only through API calls

API Documentation

- Each subsystem must provide a short API reference section in its design document
- Function purpose, parameters, return values, and side effects must be explained

---

Starter Repository Structure (Recommended)

Teams are encouraged to start from the following repository structure to keep projects organized and integration-friendly:

```
project-root/
├── README.md
├── Makefile
├── include/
│   ├── sched.h
│   ├── mem.h
│   └── sync.h
├── src/
│   ├── main.c
│   ├── sched/
│   │   └── sched.c
│   ├── mem/
│   │   └── mem.c
│   └── sync/
│       └── sync.c
├── tests/
│   └── basic_tests.c
└── docs/
    ├── api.md
    └── design.md
```

You may adapt this structure, but your repository must clearly separate **interfaces**, **implementations**, **tests**, and **documentation**.

Student FAQ – Common Failure Modes

Q1: Our subsystems work individually but fail when integrated. Why?
Most failures occur due to mismatched assumptions (data ownership, units, timing). Revisit the interface specification and validate inputs and outputs early using the vertical slice.

Q2: Can we print debug output inside API functions?
No. API functions should return status information. Logging and printing should be handled by the main system.

Q3: Our scheduler works but memory is never invoked. Is that acceptable?
No. Subsystems must interact. The vertical slice requires at least one end-to-end flow across multiple subsystems.

Q4: Do we need real concurrency or threads?
No. Logical simulation of concurrency is sufficient unless explicitly approved.

Q5: We ran out of time to integrate fully. What should we prioritize?
A correct, limited vertical slice with clear documentation is better than multiple incomplete features.

Learning Outcomes

By completing this project, students will be able to:

- Apply core operating system concepts in a practical system
- Design modular software with clear interfaces
- Analyze trade-offs in scheduling, memory, and synchronization
- Collaborate effectively on a complex technical project

Academic Integrity & Collaboration

- All code and documentation must be original
- External resources must be cited
- Team members are expected to contribute equitably
- Peer evaluations may be used to **adjust individual grades** if significant contribution imbalances are identified

---

This project is a major component of the course and is designed to reflect real-world operating system design and teamwork. Start early, communicate often, and document decisions clearly.